



Module 16: Basic Integrity and Authenticity

Networking Security v1.0
(NETSEC)



Module Objectives

Module Title: Basic Integrity and Authenticity

Module Objective: Explain how cryptography is used to ensure data integrity and authenticity.

Topic Title	Topic Objective
Integrity and Authenticity	Explain the role of cryptography in ensuring the integrity and authenticity of data.
Key Management	Describe the components of key management.
Confidentiality	Explain how cryptographic approaches enhance data confidentiality.

16.1 Integrity and Authenticity

Secure Communications

These are the four elements of secure communications:

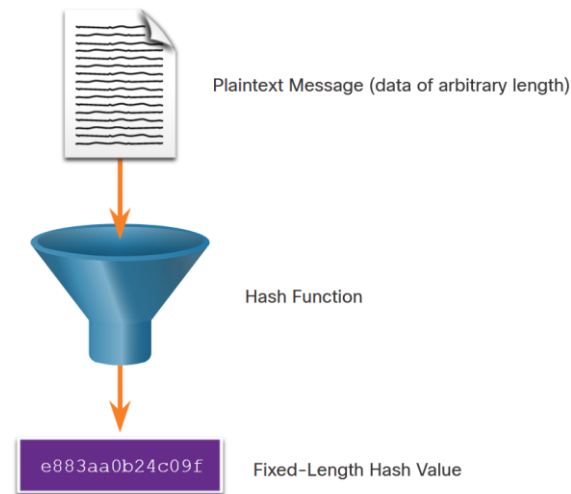
- **Data Integrity** - Guarantees that the message was not altered. Any changes to data in transit will be detected. Integrity is ensured by implementing either of the Secure Hash Algorithms (SHA-2 or SHA-3). The MD5 message digest algorithm is still widely in use. However, it is inherently insecure and creates vulnerabilities in a network. Note that MD5 should be avoided.
- **Origin Authentication** - Guarantees that the message is not a forgery and does actually come from whom it states. Many modern networks ensure authentication with algorithms such as hash-based message authentication code (HMAC).
- **Data Confidentiality** - Guarantees that only authorized users can read the message. If the message is intercepted, it cannot be deciphered within a reasonable amount of time. Data confidentiality is implemented using symmetric and asymmetric encryption algorithms.
- **Data Non-Repudiation** - Guarantees that the sender cannot repudiate, or refute, the validity of a message sent. Nonrepudiation relies on the fact that only the sender has the unique characteristics or signature for how that message is treated.

Cryptographic Hash Functions

Hashes are used to verify and ensure data integrity. Hashing is based on a one-way mathematical function that is relatively easy to compute, but significantly harder to reverse.

As shown in the figure, a hash function takes a variable block of binary data, called the message, and produces a fixed-length, condensed representation, called the hash. The resulting hash is also sometimes called the message digest, digest, or digital fingerprint.

With hash functions, it is computationally infeasible for two different sets of data to come up with the same hash output. Cryptographic hash values are often called “digital fingerprints”.

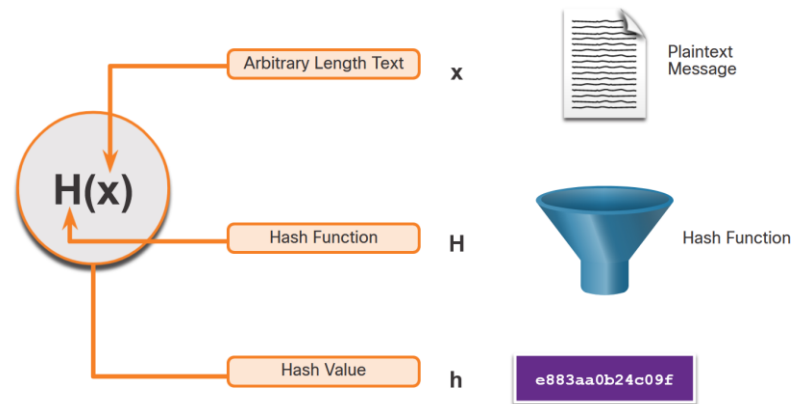


Cryptographic Hash Operation

Mathematically, the equation $h = H(x)$ is used to explain how a hash algorithm operates. As shown in the figure, a hash function H takes an input x and returns a fixed-size string hash value h .

The example in the figure summarizes the mathematical process. A cryptographic hash function should have the following properties:

- The input can be any length.
- The output is always a fixed length.
- $H(x)$ is relatively easy to compute for any given x .
- $H(x)$ is one way and not reversible.
- $H(x)$ is collision free, meaning that two different input values will result in different hash values.



MD5 and SHA

Hash functions are used to ensure the integrity of a message. They help ensure data has not accidentally changed and that what was sent is indeed what was received.

There are four well-known hash functions:

- **MD5 with 128-bit digest**
- **SHA-1**
- **SHA-2**
- **SHA-3**

While hashing can be used to detect accidental changes, it cannot be used to guard against deliberate changes that are made by a threat actor. Therefore, hashing is vulnerable to man-in-the-middle attacks and does not provide security to transmitted data. To provide integrity against man-in-the-middle attacks, origin authentication is also required.

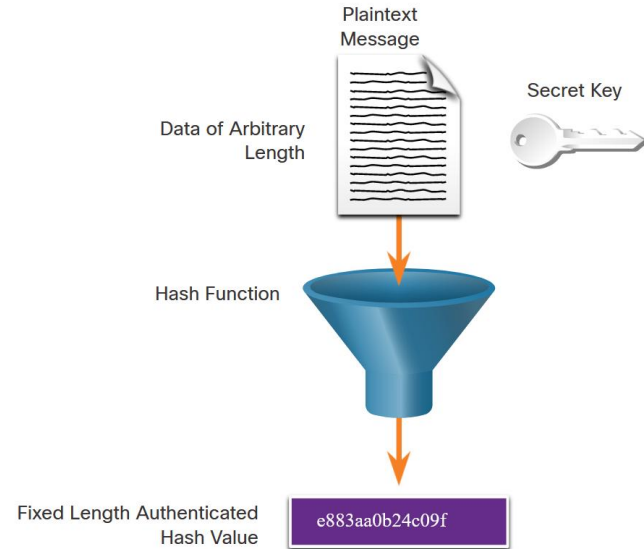
Integrity and Authenticity

Origin Authentication

To add origin authentication and integrity assurance, use a keyed-hash message authentication code (HMAC). HMAC uses an additional secret key as input to the hash function.

As shown in the figure, an HMAC is calculated using any cryptographic algorithm that combines a cryptographic hash function with a secret key. Hash functions are the basis of the protection mechanism of HMACs.

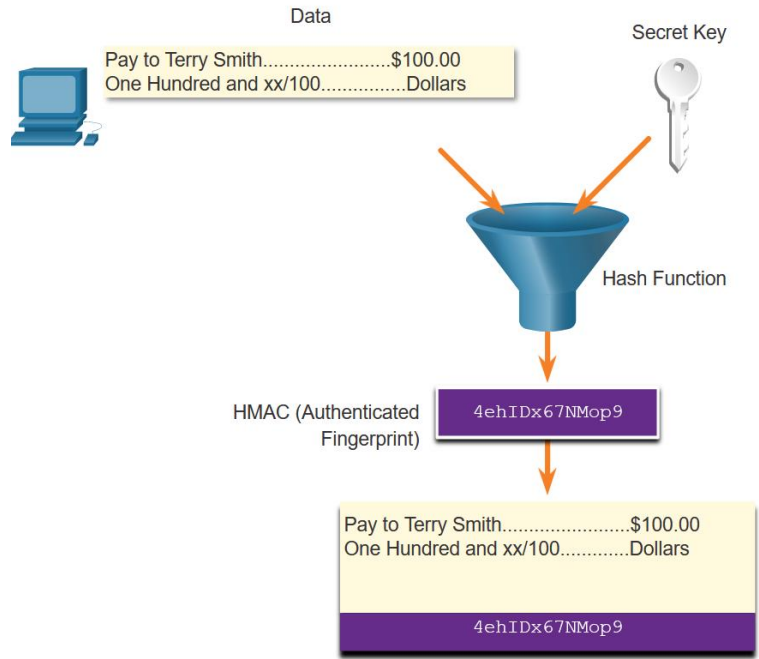
Only the sender and the receiver know the secret key, and the output of the hash function now depends on the input data and the secret key. Only parties who have access to that secret key can compute the digest of an HMAC function. This defeats man-in-the-middle attacks and provides authentication of the data origin.



Integrity and Authenticity

Origin Authentication (Cont.)

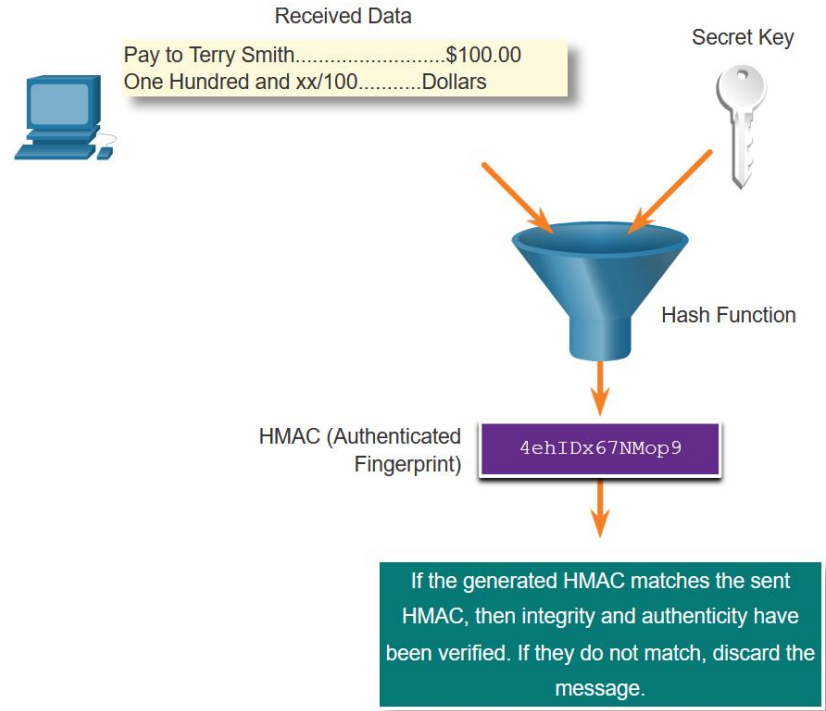
As shown in the figure, the sending device inputs data (such as Terry Smith's pay of \$100 and the secret key) into the hashing algorithm and calculates the fixed-length HMAC digest. This authenticated digest is then attached to the message and sent to the receiver.



Integrity and Authenticity

Origin Authentication (Cont.)

In the figure, the receiving device removes the digest from the message and uses the plaintext message with its secret key as input into the same hashing function. If the digest that is calculated by the receiving device is equal to the digest that was sent, the message has not been altered. Additionally, the origin of the message is authenticated because only the sender possesses a copy of the shared secret key. The HMAC function has ensured the authenticity of the message.

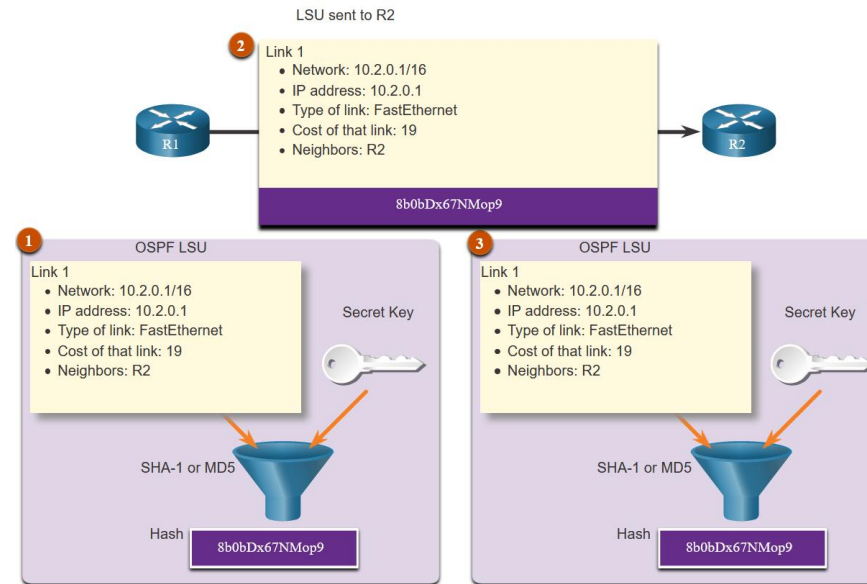


Origin Authentication (Cont.)

The figure shows how HMACs are used by Cisco routers that are configured to use Open Shortest Path First (OSPF) routing authentication.

R1 is sending a link state update (LSU) regarding a route to network 10.2.0.0/16:

1. R1 calculates the hash value using the LSU message and the secret key.
2. The resulting hash value is sent with the LSU to R2.
3. R2 calculates the hash value using the LSU and its secret key. R2 accepts the update if the hash values match. If they do not match, R2 discards the update.



Lab - Hashing Things Out

In this lab, you will complete the following objectives:

- Creating Hashes with OpenSSL
- Verifying Hashes

16.2 Key Management

Key Management

Characteristics of Key Management

Key management is often considered the most difficult part of designing a cryptosystem. As shown in the table, there are several essential characteristics of key management to consider.

Characteristic	Description
Key Generation	It was up to Caesar to choose the key of his cipher. The Vigenère cipher key is also chosen by the sender and receiver. In a modern cryptographic system, key generation is usually automated and not left to the end user. The use of good random number generators is needed to ensure that all keys are equally generated so that the attacker cannot predict which keys are more likely to be used.
Key Verification	Some keys are better than others. Almost all cryptographic algorithms have some weak keys that should not be used. With the help of key verification procedures, weak keys can be identified and regenerated to provide a more secure encryption. With the Caesar cipher, using a key of 0 or 25 does not encrypt the message, so it should not be used.
Key Exchange	Key management procedures should provide a secure key exchange mechanism that allows secure agreement on the keying material with the other party, probably over an untrusted medium.
Key Storage	On a modern multi-user operating system that uses cryptography, a key can be stored in memory. This presents a possible problem when that memory is swapped to the disk, because a Trojan horse program installed on the PC of a user could then have access to the private keys of that user.
Key Lifetime	Using short key lifetimes improves the security of legacy ciphers that are used on high-speed connections. In IPsec a 24-hour lifetime is typical. However, changing the lifetime to 30 minutes improves the security of the algorithms.
Key Revocation and Destruction	Revocation notifies all interested parties that a certain key has been compromised and should no longer be used. Destruction erases old keys in a manner that prevents malicious attackers from recovering them.

Key Length and Keyspace

Two terms that are used to describe keys are:

- **Key length** - Also called the key size, this is the measure in bits. In this course, we will use the term key length.
- **Keyspace** - This is the number of possibilities that can be generated by a specific key length.

As key length increases, the keyspace increases exponentially. The table displays the characteristics of the AES encryption algorithm.

AES Characteristics Description	Advanced Encryption Standard
Timeline	Official standard since 2001
Type of Algorithm	Symmetric
Key Size (in bits)	128, 192, and 256
Speed	High
Time to Crack(assuming a computer could try 255 keys per second)	149 trillion years
Resource Consumption	Low

The Keyspace

The keyspace of an algorithm is the set of all possible key values. A key that has n bits produces a keyspace that has 2^n possible key values. By adding one bit to the key, the keyspace is effectively doubled.

Almost every algorithm has some weak keys in its keyspace that enable an attacker to break the encryption via a shortcut. Weak keys show the regularities in encryption. For instance, DES has four keys for which encryption is the same as decryption. This means that if one of these weak keys is used to encrypt plaintext, an attacker can use the weak key to decrypt the ciphertext and reveal the plaintext.

The DES weak keys are those that produce 16 identical subkeys. This occurs when the key bits are:

- Alternating ones and zeros (0101010101010101)
- Alternating F and E (FEFEFEFEFEFEFEFE)
- E0E0E0E0F1F1F1F1
- 1F1F1F1F0E0E0E0E

Types of Cryptographic Keys

Current key lengths can easily make any attack attempt insignificant because it takes millions or billions of years to complete the search when a sufficiently long key is used.

With modern algorithms that are trusted, the strength of protection depends solely on the size of the key. Choose the key length so that it protects data confidentiality or integrity for an adequate period of time. Data that is more sensitive and needs to be kept secret longer must use longer keys.

Length of Protection	Symmetric Key	Asymmetric Key	Digital Signature	Hash
3 years	80	1248	160	160
10 years	96	1776	192	192
20 years	112	2432	224	224
30 years	128	3248	256	256
Protection against quantum computers	256	15424	512	512

Choice of Cryptographic Keys

Performance is another issue that can influence the choice of a key length. An administrator must find a good balance between the speed and protective strength of an algorithm, because some algorithms, such as the Rivest, Shamir, and Adleman (RSA) algorithm, run slowly due to large key lengths.

The estimated funding of the attacker should also affect the choice of key length.

The rule “the longer the key, the better” is valid, except for possible performance reasons. Shorter keys equal faster processing but are less secure. Longer keys equal slower processing but are more secure.

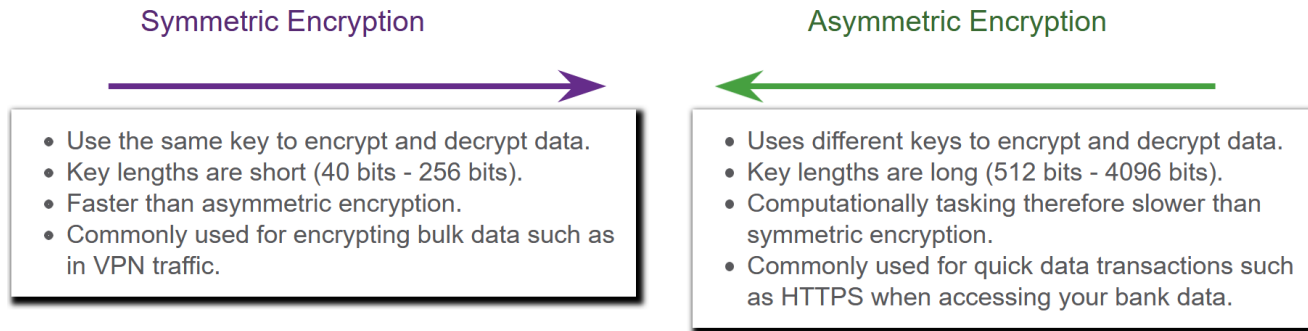
16.3 Confidentiality

Data Confidentiality

Symmetric encryption algorithms such as Data Encryption Standard (DES), 3DES, and Advanced Encryption Standard (AES) are based on the premise that each communicating party knows the pre-shared key.

Data confidentiality can also be ensured using asymmetric algorithms, including Rivest, Shamir, and Adleman (RSA) and the public key infrastructure (PKI).

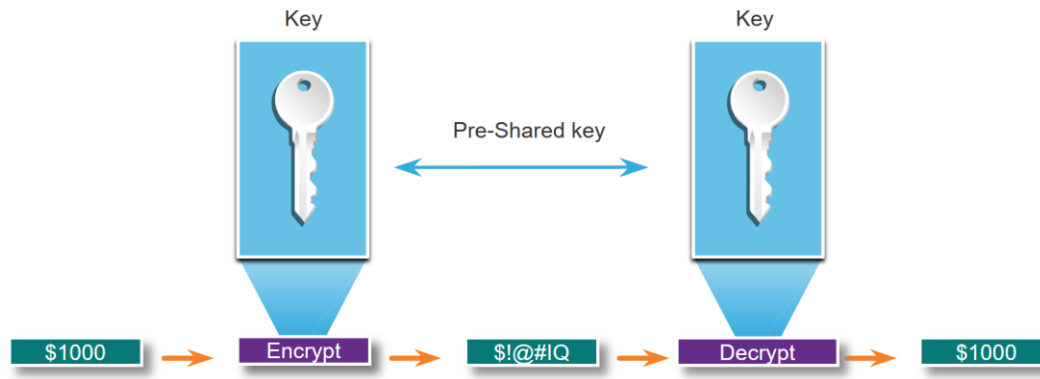
The figure highlights some differences between symmetric and asymmetric encryption



Symmetric Encryption

Symmetric algorithms use the same pre-shared key to encrypt and decrypt data. A pre-shared key, also called a secret key, is known by the sender and receiver before any encrypted communications can take place.

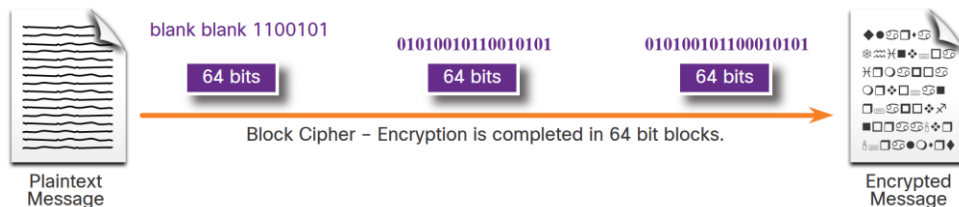
In the figure, Alice and Bob have identical keys to a single padlock. These keys were exchanged prior to sending any secret messages. Alice writes a secret message and puts it in a small box that she locks using the padlock with her key. She mails the box to Bob. The message is safely locked inside the box as the box makes its way through the post office system. When Bob receives the box, he uses his key to unlock the padlock and retrieve the message. Bob can use the same box and padlock to send a secret reply back to Alice.



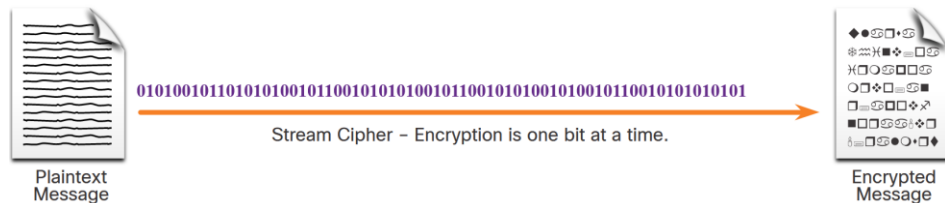
Symmetric Encryption (Cont.)

Symmetric encryption algorithms are sometimes classified as either a block cipher or a stream cipher.

Block ciphers transform a fixed-length block of plaintext into a common block of ciphertext of 64 or 128 bits. Common block ciphers include DES with a 64-bit block size and AES with a 128-bit block size.



Stream ciphers encrypt plaintext one byte or one bit at a time. Stream ciphers are basically a block cipher with a block size of one byte or bit. Stream ciphers are typically faster than block ciphers because data is continuously encrypted.



Symmetric Encryption (Cont.)

Well-known symmetric encryption algorithms are described in the table.

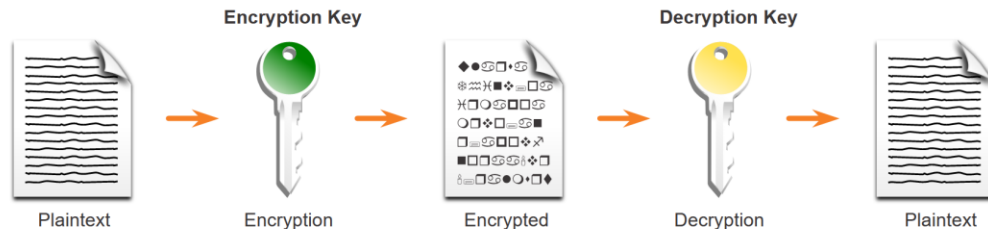
Symmetric Encryption Algorithms	Description
Data Encryption Standard (DES)	This is a legacy symmetric encryption algorithm. It uses a short key length that makes it insecure for most current uses.
3DES (Triple DES)	This is the replacement for DES and repeats the DES algorithm process three times. It should be avoided if possible as it is scheduled to be retired in 2023. If implemented, use very short key lifetimes.
Advanced Encryption Standard (AES)	AES is a popular and recommended symmetric encryption algorithm. It offers combinations of 128-, 192-, or 256-bit keys to encrypt 128, 192, or 256 bit-long data blocks.
Software-Optimized Encryption Algorithm (SEAL)	SEAL is a faster alternative symmetric encryption algorithm to AES. SEAL is a stream cypher that uses a 160-bit encryption key and has a lower impact on the CPU compared to other software-based algorithms.
Rivest ciphers (RC) series algorithms	This algorithm was developed by Ron Rivest. Several variations have been developed, but RC4 was the most prevalent in use. RC4 is a stream cipher that was used to secure web traffic. It has been found to have multiple vulnerabilities which have made it insecure. RC4 should not be used.

Asymmetric Encryption

Asymmetric algorithms, also called public-key algorithms, are designed so that the key that is used for encryption is different from the key that is used for decryption, as shown in the figure. The decryption key cannot, in any reasonable amount of time, be calculated from the encryption key and vice versa.

Examples of protocols that use asymmetric key algorithms include:

- **Internet Key Exchange (IKE)** - This is a fundamental component of IPsec VPNs.
- **Secure Socket Layer (SSL)** - This is now implemented as IETF standard Transport Layer Security (TLS).
- **Secure Shell (SSH)** - This protocol provides a secure remote access connection to network devices.
- **Pretty Good Privacy (PGP)** - This computer program provides cryptographic privacy and authentication. It is often used to increase the security of email communications.



Asymmetric Encryption (Cont.)

Asymmetric Encryption Algorithm	Key Length	Description
Diffie-Hellman (DH)	512, 1024, 2048, 3072, 4096	The Diffie-Hellman algorithm allows two parties to agree on a key that they can use to encrypt messages they want to send to each other. The security of this algorithm depends on the assumption that it is easy to raise a number to a certain power, but difficult to compute which power was used given the number and the outcome.
Digital Signature Standard (DSS) and Digital Signature Algorithm (DSA)	512 - 1024	DSS specifies DSA as the algorithm for digital signatures. DSA is a public key algorithm based on the ElGamal signature scheme. Signature creation speed is similar to RSA, but is 10 to 40 times slower for verification.
Rivest, Shamir, and Adleman encryption algorithms (RSA)	512 to 2048	RSA is for public-key cryptography that is based on the current difficulty of factoring very large numbers. It is the first algorithm known to be suitable for signing, as well as encryption. It is widely used in electronic commerce protocols and is believed to be secure given sufficiently long keys and the use of up-to-date implementations.
ElGamal	512 - 1024	An asymmetric key encryption algorithm for public-key cryptography which is based on the Diffie-Hellman key agreement. A disadvantage of the ElGamal system is that the encrypted message becomes very big, about twice the size of the original message and for this reason it is only used for small messages such as secret keys.
Elliptic curve techniques	224 or higher	Elliptic curve cryptography can be used to adapt many cryptographic algorithms, such as Diffie-Hellman or ElGamal. The main advantage of elliptic curve cryptography is that the keys can be much smaller.

Confidentiality

Asymmetric Encryption - Confidentiality

The process can be summarized using the formula:

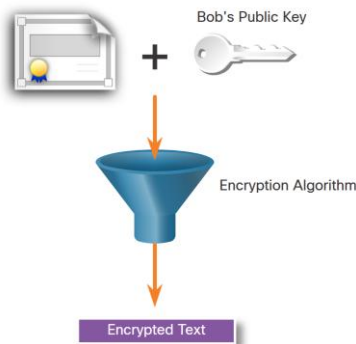
Public Key (Encrypt) + Private Key (Decrypt) = Confidentiality

When the public key is used to encrypt the data, the private key must be used to decrypt the data. Only one host has the private key; therefore, confidentiality is achieved.

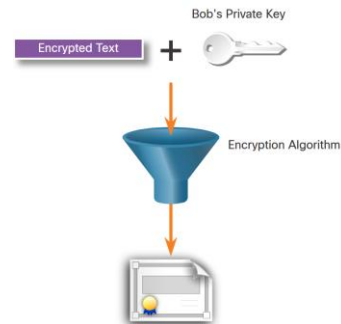
Alice requests and obtains Bob's public key.



Alice uses Bob's public key to encrypt a message using an agreed-upon algorithm. Alice sends the encrypted message to Bob.



Bob then uses his private key to decrypt the message. Since Bob is the only one with the private key, Alice's message can only be decrypted by Bob and thus confidentiality is achieved.



Confidentiality

Asymmetric Encryption - Authentication

The authentication objective of asymmetric algorithms is initiated when the encryption process is started with the private key.

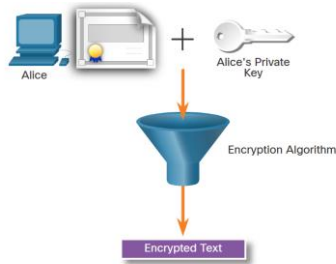
The process can be summarized using the formula:

Private Key (Encrypt) + Public Key (Decrypt) = Authentication

Alice encrypts a message using her private key. Alice sends the encrypted message to Bob. Bob needs to authenticate that the message did indeed come from Alice.

In order to authenticate the message, Bob requests Alice's public key.

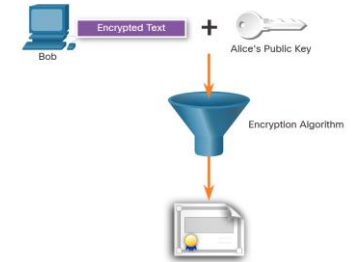
Bob uses Alice's public key to decrypt the message



Private Key (Encrypt) + Public Key (Decrypt) = Authentication



Bob needs to verify that the message actually came from Alice. He requests and acquires Alice's public key.

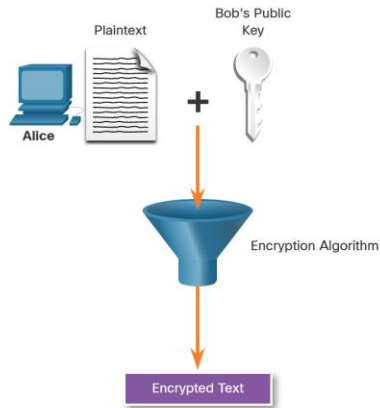


Bob uses the public key to successfully decrypt the message and authenticate that the message did, indeed, come from Alice.

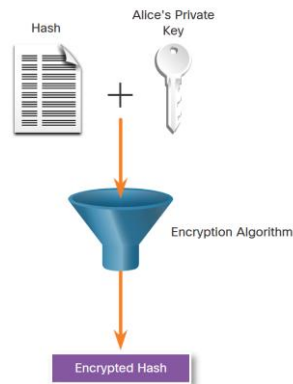
Asymmetric Encryption - Integrity

Combining the two asymmetric encryption processes provides message confidentiality, authentication, and integrity. The following example will be used to illustrate this process. In this example, a message will be ciphered using Bob's public key and a ciphered hash will be encrypted using Alice's private key to provide confidentiality, authenticity, and integrity.

Alice wants to send a message to Bob ensuring that only Bob can read the document. In other words, Alice wants to ensure message confidentiality. Alice uses the public key of Bob to cipher the message. Only Bob will be able to decipher it using his private key

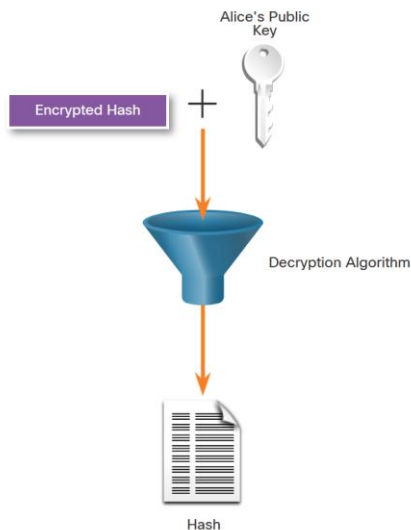


Alice also wants to ensure message authentication and integrity. Authentication ensures Bob that the document was sent by Alice, and integrity ensures that it was not modified. Alice uses her private key to cipher a hash of the message. Alice sends the encrypted message with its encrypted hash to Bob.

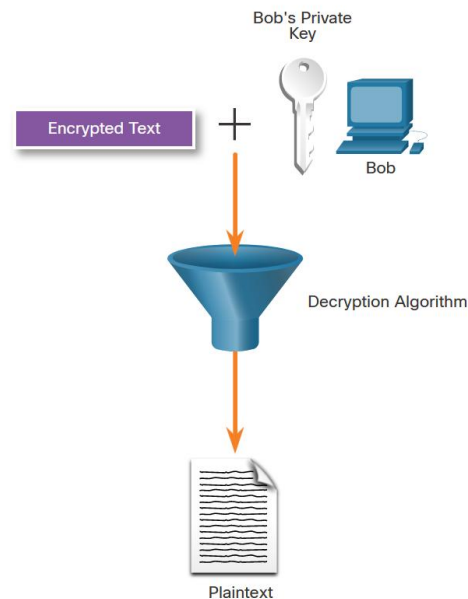


Asymmetric Encryption – Integrity (Cont.)

Bob uses Alice's public key to verify that the message was not modified. The received hash is equal to the locally determined hash based on Alice's public key. Additionally, this verifies that Alice is definitely the sender of the message because nobody else has Alice's private key.



Bob uses his private key to decipher the message.

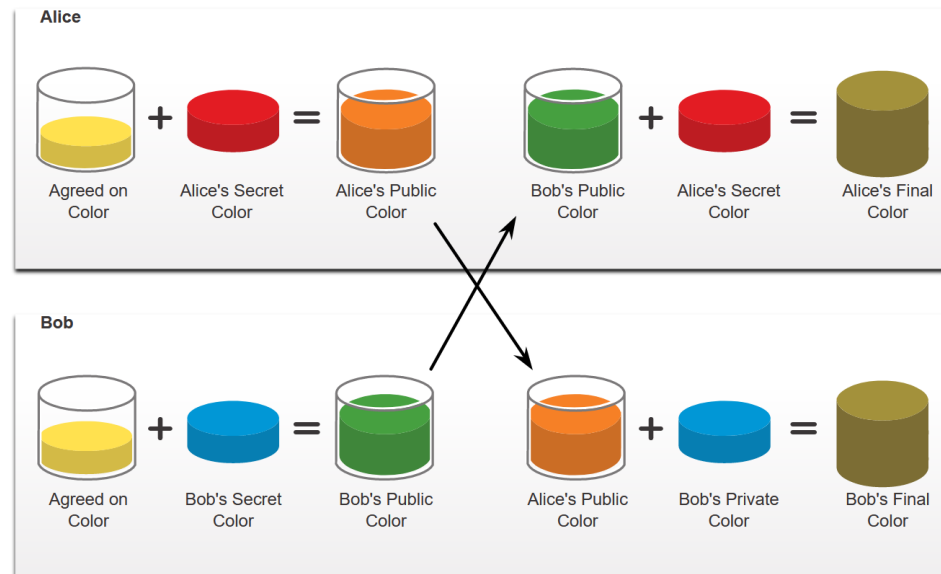


Diffie-Hellman

Diffie-Hellman (DH) is an asymmetric mathematical algorithm that allows two computers to generate an identical shared secret without having communicated before. The new shared key is never actually exchanged between the sender and receiver. However, because both parties know it, the key can be used by an encryption algorithm to encrypt traffic between the two systems. Here are two examples of instances when DH is commonly used:

- Data is exchanged using an IPsec VPN
- SSH data is exchanged

To help illustrate how DH operates, refer to the figure. The colors in the figure will be used instead of complex long numbers to simplify the DH key agreement process.



Diffie-Hellman (Cont.)

Diffie-Hellman uses different DH groups to determine the strength of the key that is used in the key agreement process. The higher group numbers are more secure but require additional time to compute the key.

The following identifies the DH groups supported by Cisco IOS Software and their associated prime number value:

DH Group 1: 768 bits

DH Group 2: 1024 bits

DH Group 5: 1536 bits

DH Group 14: 2048 bits

DH Group 15: 3072 bits

DH Group 16: 4096 bits

Video - Cryptography

This video will demonstrate securing data using hash and encryption.

Lab - Encrypting and Decrypting Data Using OpenSSL

In this lab, you will complete the following objectives:

- Encrypting Messages with OpenSSL
- Decrypting Messages with OpenSSL

Lab - Encrypting and Decrypting Data Using a Hacker Tool

In this lab, you will complete the following objectives:

- Setup Scenario
- Create and Encrypt Files
- Recover Encrypted Zip File Passwords

Lab - Examining Telnet and SSH in Wireshark

In this lab, you will complete the following objectives:

- Examine a Telnet Session with Wireshark
- Examine an SSH Session with Wireshark

16.4 Basic Integrity and Authenticity Summary

What Did I Learn in this Module?

- The four elements of secure communications are data integrity, origin authentication, data confidentiality, and data non-repudiation.
- Hashing is based on a one-way mathematical function that is relatively easy to compute, but significantly harder to reverse.
- A hash function H takes an input x and returns a fixed-size string hash value h .
- The four well-known hash functions are MD5 with 128 bit digest, SHA-1, SHA-2, and SHA-3.
- To add origin authentication and integrity assurance, use a keyed-hash message authentication code (HMAC).
- The essential characteristics of key management are key generation, key verification, key exchange, key storage, key lifetime, and key revocation and destruction.
- Two terms that are used to describe keys are key length and keyspace.
- A key that has n bits produces a keyspace that has 2^n possible key values. By adding one bit to the key, the keyspace is effectively doubled.

What Did I Learn in this Module? (Cont.)

- Cryptographic keys include symmetric keys, asymmetric keys, digital signatures, and hash keys.
- The strength of a modern algorithm depends on the size of the key.
- An administrator must find a good balance between the speed and protective strength of an algorithm.
- The two classes of encryption used to provide data confidentiality are asymmetric and symmetric.
- Symmetric encryption algorithms such as DES, 3DES, and AES are based on the premise that each communicating party knows the pre-shared key.
- Asymmetric encryption algorithms, such as RSA and PKI, are designed so that the key that is used for encryption is different from the key that is used for decryption.
- DH is commonly used when data is exchanged using an IPsec VPN and SSH data is exchanged.

New Terms and Commands

- | | |
|--|---|
| <ul style="list-style-type: none">• Secure Hash Algorithms (SHA)• MD5 message-digest algorithm• origin authentication• hash-based message authentication code (HMAC)• data non-repudiation• hash• hash function• hash value• digital fingerprint• Open Shortest Path First (OSPF)• link state update (LSU)• key management• key generation• key verification• key exchange | <ul style="list-style-type: none">• key storage• key lifetime• key revocation• key destruction• key length• keyspace• Rivest, Shamir, and Adleman (RSA) algorithm• Data Encryption Standard (DES) and 3DES• Advanced Encryption Standard (AES)• symmetric encryption• asymmetric encryption• stream ciphers• Software-Optimized Encryption Algorithm (SEAL)• Rivest ciphers (RC) series algorithms• Internet Key Exchange (IKE) |
|--|---|

New Terms and Commands (Cont.)

- Secure Socket Layer (SSL)
- Transport Layer Security (TLS)
- Pretty Good Privacy (PGP)
- Diffie-Hellman (DH)
- Digital Signature Standard (DSS)
- Digital Signature Algorithm (DSA)
- ElGamal
- Elliptic curve techniques
- public key
- private key

